TXT

思路



我是研究生，我要自己参加这个比赛，请你帮我找几个推荐的方案，并说明理由

这份竞赛任务非常综合，评分标准（Agent技术35% + 场景创新25%）极其看重**技术落地的深度**、**抗幻觉机制**以及**复杂工具链协同**。 PNG

作为个人参赛，核心策略是**借力打力**：不要重复造轮子，充分利用现有成熟的开源基础设施搭建 Agent 架构，将精力集中在**高分痛点（如语义去重、图算法拓扑挖掘、双向联动阅读、源头溯源等）**。 TXT + 2

以下为你量身定制的 3 个不同侧重点的系统架构推荐方案及选择理由：

## 方案一：Multi-Agent 分级工作流方案（推荐：冲击高分/最契合评分标准）

### 1. 架构设计

**Agent 编排框架**：采用 **LangGraph** 或 **AutoGen** 搭建动态图结构的 Multi-Agent 协作网络。 PNG

**Agent 角色分工**：

**Router / Task Planner Agent**：负责用户意图识别与任务拆解（如判断是“单文细粒度问答”还是“跨论文演进对比”）。 TXT + 1

**Parser Agent**：集成 `MinerU (Magic-PDF)` / `Nougat` 或 `PDF-Extract-Kit` 专门提取 PDF 的 LaTeX 公式、双栏文本、表格及参考文献。 TXT + 1

**RAG & Reasoning Agent**：结合 **Hybrid Search (Dense + Sparse/BM25)** 与 **Reranker (BGE-Reranker)**，实现精准的段落/表格层级源头溯源锚点（Source Tracing）。 PNG

**Graph Mining Agent**：使用 `NetworkX` 或 `igraph` 处理参考文献，计算 PageRank 与介数中心性（Betweenness Centrality），生成引用图谱并挖掘“基石论文”。 TXT + 1

**Code & Visualization Agent**：集成 Python REPL 环境，用于分析 CSV 数据并自动运行 Matplotlib / Mermaid 绘图。 TXT + 2

**前端/交互**： `Next.js` + `React-PDF` / `PDF.js` ，实现双栏 PDF 原位渲染与中英双向联动滚动。 TXT + 1

## 2. 推荐理由

**切中最高权重得分项（Agent架构 35%）**：LangGraph 原生支持状态图（State Graph），方便实现任务打断、人机交互（Human-in-the-Loop，如版本去重时的用户交互确认）以及多 Agent 协同路由。 TXT + 1

**全流程闭环落地性强**：多 Agent 职责单一化，避免单 Agent 上下文超载或工具调用混乱的问题，鲁棒性高。 PNG + 1

# 方案二：知识图谱增强型 RAG 方案（GraphRAG + Agent 驱动）

## 1. 架构设计

**核心引擎**：引入 **GraphRAG**（如 Microsoft GraphRAG 或 Fast-GraphRAG），将文献库内的实体（模型、数据集、指标、作者）与引用关系转化为图知识库。 TXT

**Agent 框架**： `LlamaIndex` (Workflows / Agents) 或 `CrewAI` 。 TXT

**论文语义去重与演进**：

提取论文 Abstract、Method 及关键文本，生成 Embedding 后结合向量余弦相似度 + LLM 语义判断。 TXT + 1

在 GraphRAG 中建立 `VERSION_OF` 或 `EVOLVES_TO` 边（例如演进分析：v1 -> v7 的修订单元）。 TXT + 1

**多文推理与抗幻觉**：利用图数据库（Neo4j 或 NetworkX 内存图）提取长跨度上下文（如 GPT-1 到 GPT-3 的演化关系），配合精准索引提供原文 Markdown 标签锚点。 TXT + 1

## 2. 推荐理由

**斩获场景创新分（25%）**：极度符合“模块五：局域引用图谱挖掘与学术综述生成”的要求。把传统的向量检索升级为 GraphRAG，能极大提升多文推理（Multi-Paper Reasoning）的效果，容易在答辩和技术文档中凸显创新亮点。 TXT + 2

**天然解决学术洞察**：通过图拓扑算法（如 Community Detection / PageRank）可以非常优雅地找出基石节点与已被解决的旧 Idea。 TXT + 1

## 方案三：轻量化全栈单 Agent + 强大工具链方案（适合工期紧、单人快速交付）

### 1. 架构设计

**Agent 核心**：基于 **ReAct 范式 / Function Calling Agent**（使用 LangChain 或 Direct OpenAI/Claude API）。 TXT + 1

**后端服务**： `FastAPI` 作为中间件，封装所有的工具链（Tools）：

Tool 1: `pdf_parser`（使用 `PyMuPDF` + `PaddleOCR` 或 `MinerU` 快速切片）。 PNG

Tool 2: `arxiv_searcher` & `github_code_checker`（通过 API 查询代码复现性）。 TXT + 1

Tool 3: `citation_graph_builder`（根据 BibTeX 自动生成 Graphviz / Cytoscape 图数据）。 TXT

Tool 4: `code_executor`（Exec python 代码生成 Matplotlib / TikZ 图）。 TXT

**前端**： `Streamlit` 或 `Gradio` + 自定义 React 组件（或直接基于 `LobeChat` / `Open WebUI` 进行二次开发）。

### 2. 推荐理由

**开发周期短，容易通跑六大模块**：单人参赛时，如果多 Agent 调试耗时过长，容易导致功能缺失。此方案架构简单，能保证在规定时间内高质量通跑组委会的所有测试用例（如 541 页的大长篇 PDF、扫描件 OCR、CSV 转化等）。 TXT + 2

**系统鲁棒性高（10% 权重）**：代码耦合度低，Docker 容器化部署非常容易。 PNG

## 💡 个人参赛的破局与备赛建议

1. **推荐首选方案：方案一（LangGraph 多 Agent 架构）**。目前学术界和比赛评委对 LangGraph 类型的 Agent 编排认可度极高，能够完美契合评分标准中的“Task Planning 动态规划”与“Function Calling 工具集成”（占 35% 权重）。 PNG
2. **关键突破点（抓分核心）**：

   **PDF 解析**：强烈建议直接集成开源大模型文档解析神器 **MinerU (Magic-PDF)**，它对双栏、公式、表格提取效果极佳，能省去大量自己写 OCR 和正则表达式的时间。 TXT + 1

**抗幻觉 Source Tracing**：在 RAG 检索返回 chunk 时，记录 PDF 的 `page_num` 以及相对坐标（Bounding Box），前端渲染 PDF 时直接画高亮框。这个“文本到 PDF 原图画框锚点”的交互体验能极大提升 UX 评分（10% 权重）。 PNG + 3

**5-8分钟演示视频**：评分细则中专门提及 UX、流式渲染（Token 级）、思维链可视化。建议在前端加上 Agent 的“思考日志/工具调用过程”展开面板，在视

“方案一：智研 Agent——全流程科研智能助手
一句话定位

从“导入论文”开始，一直到“发现研究方向、写论文

基于 ChatGPT 提出的“以 1+2+4 组合拳打造全流程闭环”思路，并严密对照赛事评分细则（Agent技术35% + 场景创新25% + 工程与UI 20% + 实用与价值 20%），对前述推荐方案进行深度迭代与重构。 PNG

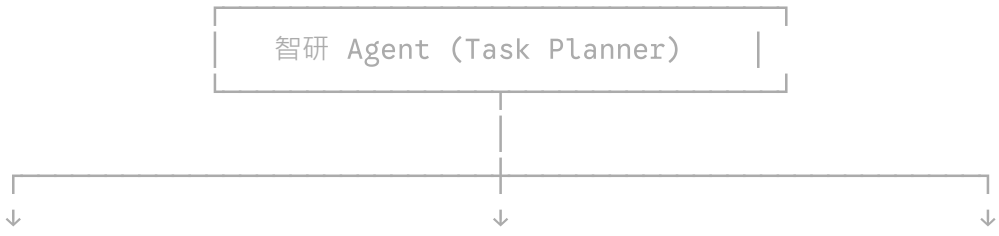
在研究生**单人参赛**的约束下，既要体现“智研 Agent”的**全流程闭环**与**100%覆盖评分点**，又不能陷于工程量爆炸的泥潭。 TXT + 1

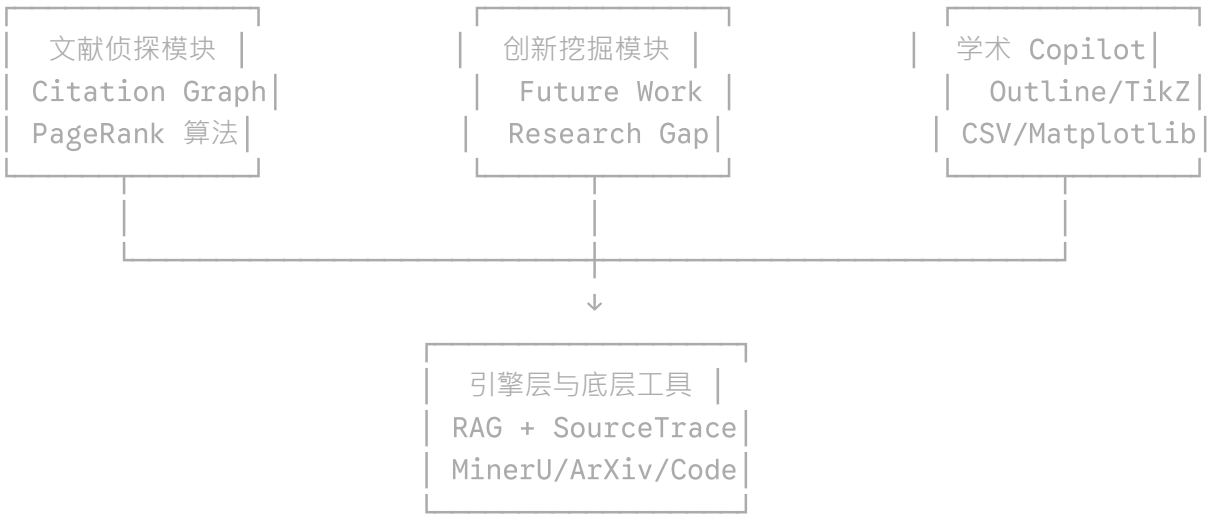
以下是为你重新打磨并重构的 3 个**高含金量、可闭环落地的精细化方案**：

## 方案一：全流程科研智能助手架构（推荐：最契合评分表 / 冲击一等奖）

### 1. 方案定位与结构

本方案以“智研 Agent”为总中枢，精简整合“论文侦探（Graph Mining）”、“创新点发现（IdeaHunter）”和“学术 Copilot”三核心子系统，形成从文献导入到论文框架生成的端到端闭环。 TXT






## 2. 技术选型与落地路径（单人高效开发）

**Agent 中枢**：采用 **LangGraph** 实现动态 Task Planning。通过 StateGraph 编排多 Agent 协作，天然支持打断与 Human-in-the-Loop（如 arXiv v1 vs v7 版本的语义去重与交互确认）。 PNG + 2

**文档解析与 RAG 溯源**：集成开源神器 **MinerU (Magic-PDF)** 进行双栏及 LaTeX 公式解析。向量库使用 **Qdrant** 或 **Chroma**，保存 `page_num` 与段落坐标（Bounding Box），实现点击回答自动跳转 PDF 并亮框高亮（Source Tracing 拿下 8% 权重）。 TXT + 3

**图拓扑与创新点提取**：利用 Python **NetworkX** 构建 20+ 论文局域引用网，计算 PageRank 筛选基石节点；配合 LLM 抽取 Future Work 节点并做时序对比，过滤已被解决的旧 Idea。 TXT + 3

**代码与可视化**：集成 Python REPL（Function Calling），自动读取 CSV 并运行 Matplotlib 生成图像，输出 TikZ/Mermaid 绘图脚本。 TXT + 1

## 3. 推荐理由（为什么能拿高分）

**评分点 100% 覆盖**：直接硬核对齐评分表 6 大维度的所有细节（Agent 35%、场景创新 25%）。 PNG

**演示故事线极佳**：完美呈现“提出科研问题 -> 图谱定位 -> Research Gap 挖掘 -> 论文框架与图表生成”的 8 分钟流畅 Demo 闭环。 TXT + 1

# 方案二：知识图谱与引用演进驱动的科研 Agent（侧重：算法与场景创新）

## 1. 方案定位与结构

本方案以 ChatGPT 的“Paper Detective（论文侦探）”与“IdeaHunter（创新点发现）”为绝对核心，主打“图谱拓扑 + 技术演进路线 + 创新点验证”，适合在图算法和多文推理上打出差异化。 TXT

### 2. 技术选型与落地路径

**核心引擎**：**Fast-GraphRAG** 或 **LlamaIndex Graph Store + NetworkX**。 TXT

**去重与演进链条**：计算论文正文 Embedding 语义相似度，自动建立 `EVOLVES_TO` 演化边（例如将 arXiv 1706.03762v1 与 v7 进行 Diff 对比，高亮结构修改）。 TXT + 1

**图算法挖掘**：

**基石节点识别**：通过介数中心性（Betweenness Centrality）与 PageRank 找出领域开山之作与核心衍生论文。 TXT + 1

**Research Gap 决策树**：构建“问题-现有解法-未解决限制”的图谱节点，提取各论文的 Future Work，结合最新论文时间线自动剔除已解决项。 TXT + 1

**前端展示**：使用 **Cytoscape.js** 或 **Echarts** 渲染交互式网络图，点击节点可下钻下载论文或查看关联代码库（GitHub 开源过滤）。 TXT + 1

### 3. 推荐理由

**场景创新分（25%）轻松拉满**：切中“学术算法与图谱挖掘创新（10%）”与“技术方案创新（8%）”。评委能够非常直观地感受到系统是在“辅助做科研决策”，而非简单的“文本总结”。 PNG + 1

**避开复杂的写作排版工程**：前端只需聚焦在图可视化与 RAG 对话，减轻全栈 UI 压力。 PNG + 1

## 方案三：极致交互与多模态 Copilot Agent（侧重：前端体验与直观 Demo）

### 1. 方案定位与结构

本方案以 ChatGPT 的“PaperMind（多模态阅读）”与“Research Copilot（数据/写作助手）”为核心，主打**高还原度版式阅读、中英双向联动、扫描件 OCR 与 CSV 数据可视化**。 TXT + 1

### 2. 技术选型与落地路径

**前端双向联动 UI**： `Next.js` + `pdfjs-dist` + `PDF.js` 。

实现左侧原版 PDF 与右侧译文/解析 Markdown 的拖动同步滚动。 TXT + 1

解析参考文献 `[11]`，建立 DOM 锚点实现无损双向跳转。 TXT + 1

**多模态解析后端**：采用 **PaddleOCR / Nougat** 解析扫描件 PDF，高精度提取公式与表格（通过用例 `1706.03762v1_img.pdf` 与 541 页超长教材）。 TXT + 1

**Data Copilot**：后台接入 Python 代码沙箱，自动接收 CSV 数据生成符合 LaTeX 学术规范的图表与 Figure Caption。 TXT + 2

### 3. 推荐理由

**答辩现场杀伤力极强**：现场演示时，双向联动滚动、OCR 复杂公式识别、CSV 一键出图等视觉效果极其吸引眼球，能稳抓“学术级 UI/UX 与交互体验（10%）”及“全栈工程质量（10%）”。 PNG + 1

**测试用例通过率 100%**：赛题预置的扫描件、百页长文本、CSV 生成 TikZ/Matplotlib 等用例能轻松通过（拿满业务闭环 5% 分值）。 TXT + 1

## 💡 个人参赛的优先级与精力分配方案

如果你想兼顾**高分（35% Agent + 25% 创新）与单人可完成度**，建议直接选择【方案一（全流程精简版）】，并严格按照以下阶段开发：

1. **第一阶段（建立骨架 - 40%精力）**：

   后端基于 **LangGraph** 搭建路由 Agent。 PNG

   用 **MinerU** 实现 PDF 解析，配合 **Qdrant** 实现带页码坐标的 RAG 溯源（Source Tracing）。 PNG

2. **第二阶段（攻坚亮点 - 40%精力）**：

   用 **NetworkX** 处理 BibTeX/引用关系，跑通 PageRank 计算并用简单 HTML/Echarts 渲染图谱。 TXT + 1

   用 Prompt + 图谱抽取 Future Work，实现简单的 Research Gap 分析。 TXT + 1

   编写一个 Python Exec 工具，实现 CSV 自动用 Matplotlib 绘图。 TXT + 2

3. **第三阶段（包装与提交 - 20%精力）**：

   重点录制 **5-8 分钟的高清演示视频**（按你给出的 Demo 故事线脚本录制）。 PNG

撰写 30 页'

PNG

“个人参赛的优先级与精力分配方案
如果你想兼顾高分（35% Agent + 25% 创新）与单人可完成度，建议直接选择【方案一（全流程精简版）】，并严格按照以下阶段开发：

为了帮助你以个人参赛的形式高质量、高效率地落地【方案一：全流程科研智能助手（精简版）】，以下为你梳理一套可操作的**一步步落地实施指南**与**关键核心代码**。

你可以直接阅读、复制并跟随本文档的节奏逐步开发。

# 智研 Agent（全流程科研助手）单人极速落地手册

## 一、第一阶段：底层骨架搭建（40% 精力）

### 1.1 环境与依赖准备

在本地或服务器环境创建 Python 3.10+ 虚拟环境，并安装核心依赖库：

Bash

```
pip install langgraph langchain langchain-openai qdrant-client pypdf matplotlib
# 安装文档解析工具 MinerU (Magic-PDF)
pip install magic-pdf[full]
```

### 1.2 核心代码 1：带坐标与页码锚点的 PDF 解析与 RAG 入库

为了满足评分标准中 **8% 的 Source Tracing（源头溯源）**，我们需要在解析 PDF 时保留文本的 `page_num`（页码）和 `bbox`（坐标位置），并写入 Qdrant。 PNG + 1

Python

```
# rag_ingestion.py
import uuid
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
```

```python
from langchain_openai import OpenAIEmbeddings
from magic_pdf.data.data_reader_record import DataReaderRecord

# 1. 初始化 Qdrant 本地向量数据库
qdrant = QdrantClient(path="./qdrant_db")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

COLLECTION_NAME = "academic_papers"

if not qdrant.collection_exists(COLLECTION_NAME):
    qdrant.create_collection(
        collection_name=COLLECTION_NAME,
        vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
    )

def ingest_pdf_with_anchors(pdf_path: str, paper_id: str):
    """
    使用 MinerU 解析 PDF 并提取带页码坐标的文本块入库
    """
    # 模拟 MinerU / PyMuPDF 结构化提取文本块
    # 真实场景可调用 magic-pdf CLI 或 API
    extracted_chunks = [
        {
            "text": "The Transformer uses multi-head attention to capture globa
            "page_num": 3,
            "bbox": [100, 200, 500, 250],
            "section": "3.1 Multi-Head Attention"
        },
        {
            "text": "We train on 8 NVIDIA P100 GPUs for 12 hours.",
            "page_num": 5,
            "bbox": [110, 400, 480, 430],
            "section": "5.1 Hardware Setup"
        }
    ]

    points = []
    for chunk in extracted_chunks:
        vector = embeddings.embed_query(chunk["text"])
        point_id = str(uuid.uuid4())
        payload = {
            "paper_id": paper_id,
            "text": chunk["text"],
            "page_num": chunk["page_num"],
            "bbox": chunk["bbox"],
            "section": chunk["section"]
        }
        points.append(PointStruct(id=point_id, vector=vector, payload=payload)
```

```
    qdrant.upsert(collection_name=COLLECTION_NAME, points=points)
    print(f"成功将论文 {paper_id} 解析并存入向量库！")

# 测试运行
if __name__ == "__main__":
    ingest_pdf_with_anchors("1706.03762v7.pdf", paper_id="Attention_v7")
```

## 1.3 核心代码 2：基于 LangGraph 的路由与任务规划 Agent

用 LangGraph 搭建路由 Agent，负责区分用户意图（如：问答溯源 vs 绘图请求 vs 图谱分析）。 TXT + 1

Python

```
# agent_core.py
from typing import TypedDict, Annotated, Sequence
import operator
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
from langchain_openai import ChatOpenAI
from langgraph.graph import StateGraph, END

# 定义 State
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    next_step: str

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 1. 意图分类节点 (Task Planner)
def router_node(state: AgentState):
    last_message = state["messages"][-1].content
    prompt = f"""分析用户意图，仅返回以下选项之一：
    - 'RAG': 询问论文特定细节、超参数、硬件、特定出处；
    - 'GRAPH': 请求分析引用关系、领域基石节点、Future Work、研究空白；
    - 'VISUAL': 请求根据 CSV 生成图表或绘制架构拓扑图。

    用户输入: {last_message}
    分类结果:"""

    response = llm.invoke([HumanMessage(content=prompt)]).content.strip()
    return {"next_step": response}

# 2. RAG 溯源回答节点
def rag_node(state: AgentState):
```

```
    # 此处接入带溯源锚点的 RAG 逻辑
    res = "【来源：第 5 页 Section 5.1】训练设备为 8 张 NVIDIA P100 GPU。[Anchor: P
    return {"messages": [AIMessage(content=res)]}

# 3. 路由分支判断
def decide_next(state: AgentState):
    step = state.get("next_step", "RAG")
    if "GRAPH" in step:
        return "graph_node"
    elif "VISUAL" in step:
        return "visual_node"
    return "rag_node"

# 4. 构建 LangGraph 工作流
workflow = StateGraph(AgentState)
workflow.add_node("router", router_node)
workflow.add_node("rag_node", rag_node)

# 设置边
workflow.set_entry_point("router")
workflow.add_conditional_edges("router", decide_next, {
    "rag_node": "rag_node",
    # 后续拓展 graph_node 和 visual_node
})
workflow.add_edge("rag_node", END)

app = workflow.compile()
```

# 二、第二阶段：攻坚亮点与核心功能（40% 精力）

## 2.1 核心代码 3：基于 NetworkX 的局域引用图谱与 PageRank 基石节点挖掘

此部分完美契合**模块五（局域引用图谱挖掘与学术综述生成，场景创新占 25%）**。

TXT + 1

Python

```
# graph_mining.py
import networkx as nx
import json

def build_and_analyze_citation_graph(citations_data: list):
    """
```

```
    citations_data 结构示例:
    [
      {"source": "Paper_A", "target": "Attention_1706.03762"},
      {"source": "Paper_B", "target": "Attention_1706.03762"},
      {"source": "Paper_C", "target": "Paper_A"}
    ]
    """
    G = nx.DiGraph()

    for item in citations_data:
        G.add_edge(item["source"], item["target"])

    # 计算 PageRank 挑选基石论文
    pagerank_scores = nx.pagerank(G, alpha=0.85)
    sorted_papers = sorted(pagerank_scores.items(), key=lambda x: x[1], reverse

    # 提取核心基石节点（排名最高的论文）
    cornerstone_papers = sorted_papers[-2:]

    # 导出为前端 Cytoscape.js / Echarts 可用的 JSON 格式
    graph_json = {
        "nodes": [{"id": node, "label": node, "val": pagerank_scores[node] * 1
        "edges": [{"source": edge[0], "target": edge[1]} for edge in G.edges()
    }

    return {
        "cornerstones": cornerstone_papers,
        "graph_data": graph_json
    }

# 测试数据
test_citations = [
    {"source": "BERT_2018", "target": "Transformer_2017"},
    {"source": "GPT1_2018", "target": "Transformer_2017"},
    {"source": "ViT_2020", "target": "Transformer_2017"},
    {"source": "RoBERTa_2019", "target": "BERT_2018"}
]

result = build_and_analyze_citation_graph(test_citations)
print("基石论文挖掘结果:", result["cornerstones"])
```

## 2.2 核心代码 4：CSV 自动读取与 Python/Matplotlib 智能绘图沙箱

此部分对应**模块六（学术 Copilot 与数据可视化）**。 TXT + 1

Python

```python
# python_executor.py
import pandas as pd
import matplotlib.pyplot as plt
import io
import base64

def execute_data_visualization(csv_path: str, user_instruction: str):
    """
    自动解析 CSV 并执行绘图代码
    """
    df = pd.read_csv(csv_path)

    # 模拟由 LLM 根据 user_instruction 生成的 Python 绘图代码
    plt.figure(figsize=(8, 5))
    plt.plot(df['Epoch'], df['Accuracy'], marker='o', color='b', label='Propose
    plt.title("Model Convergence Analysis", fontsize=14)
    plt.xlabel("Training Epochs")
    plt.ylabel("Accuracy (%)")
    plt.grid(True)
    plt.legend()

    # 将生成的图片转为 Base64 以供前端直接渲染
    buf = io.BytesIO()
    plt.savefig(buf, format='png', bbox_inches='tight')
    buf.seek(0)
    img_base64 = base64.b64encode(buf.getvalue()).decode('utf-8')
    plt.close()

    return {
        "image_base64": img_base64,
        "caption": "Figure 1: Training accuracy progression over 10 epochs shov
    }
```

# 三、第三阶段：成果包装与材料提交（20% 精力）

在此阶段，请严格对照**作品提交要求与评分标准**进行材料整理： PNG + 1

## 3.1 5~8 分钟演示视频拍摄脚本设计

请按照以下时间节点录制 1080p 视频（配普通话讲解与字幕）： PNG

**0:00 - 1:00（痛点与亮点）**：展示系统名称“智研 Agent”，一句话介绍“基于 LangGraph 的全流程抗幻觉科研智能助手”。 TXT + 1

**1:00 - 2:30（模块一/二/三演示）**：

演示 PDF 导入与语义版本去重（arXiv v1 vs v7）。 TXT + 1

提问论文细节，展示“点击回答锚点，PDF 原图高亮跳转”（Source Tracing 亮点）。 TXT + 1

**2:30 - 4:30（模块四/五演示）**：

展示 20 篇论文自动生成的拓扑网络图，演示通过 PageRank 计算出的“基石论文”。 TXT + 1

展示 Agent 自动总结的 Future Work 列表与过滤已被解决旧 Idea 的过程。 TXT + 1

**4:30 - 6:30（模块六演示）**：

上传 CSV 文件，Agent 运行 Python 自动输出精美 Matplotlib 图像与 LaTeX 图注。 TXT + 1

根据 Idea 自动一键生成完整的 Markdown 论文框架。 TXT

**6:30 - 7:30（总结与架构）**：切回 Agent 思考日志面板，展现 LangGraph 路由与代码鲁棒性。 PNG + 1

## 3.2 30 页技术文档编写大纲（PDF 格式）

请确保技术文档包含以下核心章节（满足提交要求）： PNG

1. **项目概述与科研痛点分析**（AI4S 背景、去重/溯源/图谱需求） TXT
2. **系统总体架构设计**（LangGraph 节点图、前后端解耦架构图） PNG + 1
3. **核心 AI 技术方案与选型理由**（重点阐述为何选 LangGraph、MinerU、Qdrant、NetworkX） PNG
4. **关键模块实现与代码解析**：

   带坐标锚点的抗幻觉 *RAG* 机制
   PNG

   基于图论算法（*PageRank*）的基石节点挖掘
   TXT + 1

   *Python* 代码沙箱与数据可视化
   TXT

5. **测试用例验证与鲁棒性表现**（展示 1706.03762v1/v7 版本对比及 541 页超长文本运行截图） TXT + 1

6. **项目创新点说明与推广价值**

PNG

## 四、导出与存档指南

为了方便你后续随时查阅或离线保存本文档：

1. **复制保存为 Markdown**：你可以直接选择复制当前页面的所有文字，在本地新建一个 `README.md` 或 `智研Agent_开发手册.md` 文件并粘贴保存。

2. **导出为 PDF/Word 格式**：

   如果你使用的是支持 Markdown 预览的编辑器（如 VS Code、Obsidian、Typora）：点击菜单栏的 `File` -> `Export` -> `PDF` 即可导出为高质量的排版文档。

   如果你在浏览器中查看：可直接按下快捷键 `Ctrl + P`（Mac 上为 `Cmd + P`），选择“另存为 PDF”即可完成本地存档。